1) Two Sum

Problem statement
Given an integer array nums and an integer target, return indices of the two numbers such that they add up to target. You may assume each input would have exactly one solution, and you may not use the same element twice. Return indices in any order.

What interviewer tests
Hashing basics, single-pass thinking, O(n) solution vs naive $O(n^2)$. Edge-case handling and index tracking.

Approach (optimal)
Use a HashMap mapping value -> index. For each element x at index i, check if target - x exists in the map. If yes, we found a solution; otherwise store x -> i. This is single pass O(n).

Algorithm (step-by-step)

- Create map valueToIndex = new HashMap<Integer,Integer>().
- Iterate i = 0..n-1:
    - complement = target - nums[i]
    - if map contains complement: return [map.get(complement), i]
    - else put nums[i] -> i into map
- (Assuming exactly one solution always exists as per prompt.)

Java code

```
public int[] twoSum(int[] nums, int target) {
  Map<Integer, Integer> valueToIndex = new HashMap<>();
  for (int i = 0; i < nums.length; i++) {
    int complement = target - nums[i];
    if (valueToIndex.containsKey(complement)) {
      return new int[]{ valueToIndex.get(complement), i };
    }
    valueToIndex.put(nums[i], i);
  }
```

```
    throw new IllegalArgumentException("No two sum solution");
}
```

Complexity

- Time: O(n) — single pass.
- Space: O(n) — HashMap.

Example
nums = [2,7,11,15], target = 9
Iteration: i=0 store 2->0; i=1 complement=2 found -> return [0,1].

---

2) Best Time to Buy and Sell Stock I

Problem statement
Given array prices[] where prices[i] is price of a given stock on day i, find the maximum profit you can achieve by choosing a single day to buy and a later day to sell. If no profit is possible return 0.

What interviewer tests
Greedy one-pass idea, tracking min-so-far and max-profit.

Approach (optimal)
Track minPrice encountered so far and maxProfit. For each price, consider selling today: profit = price - minPrice; update maxProfit; update minPrice if current price is smaller.

Algorithm (step-by-step)

- Initialize minPrice = Integer.MAX_VALUE, maxProfit = 0.
- For each price p:
    - if p < minPrice, minPrice = p
    - else if p - minPrice > maxProfit, update maxProfit
- Return maxProfit

Java code

```
public int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE;
```

```
    int maxProfit = 0;
    for (int p : prices) {
        if (p < minPrice) minPrice = p;
        else if (p - minPrice > maxProfit) maxProfit = p - minPrice;
    }
    return maxProfit;
}
```

Complexity

- Time: O(n)
- Space: O(1)

Example
[7,1,5,3,6,4] -> minPrice=1, maxProfit=5 (buy at 1 sell at 6).

---

3) Maximum Subarray (Kadane's Algorithm)

Problem statement
Given integer array nums, find contiguous subarray (containing at least one number) which has the largest sum and return its sum.

What interviewer tests
Dynamic programming idea condensed into greedy running-sum; ability to convert recurrence to O(1) space.

Approach (optimal)
Kadane's: maintain currentSum = max subarray sum ending at current index; update maxSum globally. Recurrence currentSum = max(nums[i], currentSum + nums[i]).

Algorithm (step-by-step)

- Initialize maxSum = nums[0], currentSum = nums[0].
- For i from 1..n-1:
    - currentSum = Math.max(nums[i], currentSum + nums[i])
    - maxSum = Math.max(maxSum, currentSum)

- Return maxSum

Java code

```
public int maxSubArray(int[] nums) {
  int currentSum = nums[0];
  int maxSum = nums[0];
  for (int i = 1; i < nums.length; i++) {
    currentSum = Math.max(nums[i], currentSum + nums[i]);
    maxSum = Math.max(maxSum, currentSum);
  }
  return maxSum;
}
```

Complexity

- Time: O(n)
- Space: O(1)

Example
[-2,1,-3,4,-1,2,1,-5,4] -> max sum = 6 (subarray [4,-1,2,1]).

---

4) Product of Array Except Self

Problem statement
Given an integer array nums, return an array answer such that answer[i] is product of all elements of nums except nums[i]. Solve in O(n) time without using division and with O(1) extra space (output array excluded).

What interviewer tests
Prefix/suffix product technique, space optimization and careful order of operations.

Approach (optimal)
Compute left prefix products into answer then multiply by running right product. Use answer to store left products first, then multiply by right cumulative product iterating from right to left.

Algorithm (step-by-step)

- Let n = nums.length.
- Create answer = new int[n].
- Compute left products: answer[0] = 1; for i=1..n-1: answer[i] = answer[i-1] * nums[i-1].
- Maintain rightProduct = 1; for i=n-1..0: answer[i] *= rightProduct; rightProduct *= nums[i];
- Return answer.

Java code

```
public int[] productExceptSelf(int[] nums) {
  int n = nums.length;
  int[] answer = new int[n];
  answer[0] = 1;
  for (int i = 1; i < n; i++) {
    answer[i] = answer[i - 1] * nums[i - 1];
  }
  int rightProduct = 1;
  for (int i = n - 1; i >= 0; i--) {
    answer[i] = answer[i] * rightProduct;
    rightProduct *= nums[i];
  }
  return answer;
}
```

Complexity

- Time: O(n)
- Space: O(1) extra (output not counted)

Edge cases
Zeros: method handles zeros correctly (no division used).

Example
[1,2,3,4] -> left pass answer=[1,1,2,6], right multiply gives [24,12,8,6].

---

5) Longest Substring Without Repeating Characters

Problem statement
Given a string s, find the length of the longest substring without repeating characters.

What interviewer tests
Sliding-window + index tracking; careful window movement.

Approach (optimal)
Sliding window with map of char -> last index seen. Keep two pointers left and right. When encountering duplicate inside window, move left to max(left, lastIndex + 1).

Algorithm (step-by-step)

- Use Map<Character, Integer> lastIndex.
- left = 0; maxLen = 0.
- For right = 0..n-1:
    - if s[right] seen before: left = Math.max(left, lastIndex.get(s[right]) + 1)
    - update lastIndex.put(s[right], right)
    - update maxLen = Math.max(maxLen, right - left + 1)

Java code

```
public int lengthOfLongestSubstring(String s) {
  Map<Character, Integer> lastIndex = new HashMap<>();
  int left = 0, maxLen = 0;
  for (int right = 0; right < s.length(); right++) {
    char c = s.charAt(right);
    if (lastIndex.containsKey(c)) {
      left = Math.max(left, lastIndex.get(c) + 1);
```

```
        }
        lastIndex.put(c, right);
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Complexity

- Time: O(n)
- Space: O(min(n, charset)) — map for last indices.

Example
s = "abcabcbb" -> result 3 ("abc").

---

6) Longest Palindromic Substring

Problem statement
Given s, return the longest palindromic substring.

What interviewer tests
Center-expand technique, or DP/Manacher's algorithm for optimal. Expect center expand for interview.

Approach (center expand)
Every palindrome is centered at a character (odd length) or between two characters (even length). Expand from each center and track the longest found.

Algorithm (step-by-step)

- For each index i:
    - Expand odd center: left=i, right=i
    - Expand even center: left=i, right=i+1
    - If expanded length > best, record start/end.
- Return substring.

Java code

```
public String longestPalindrome(String s) {
   if (s == null || s.length() < 1) return "";
   int start = 0, end = 0;
   for (int i = 0; i < s.length(); i++) {
      int len1 = expandAroundCenter(s, i, i);
      int len2 = expandAroundCenter(s, i, i + 1);
      int len = Math.max(len1, len2);
      if (len > end - start + 1) {
         start = i - (len - 1) / 2;
         end = i + len / 2;
      }
   }
   return s.substring(start, end + 1);
}

private int expandAroundCenter(String s, int left, int right) {
   while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
      left--; right++;
   }
   return right - left - 1; // length of palindrome
}
```

Complexity

- Time: $O(n^2)$ worst-case (all characters same)
- Space: O(1) (excluding output)

Example

s = "babad" -> return "bab" or "aba".

---

7) Container With Most Water

Problem statement
Given height[] of n vertical lines at x=0..n-1, find maximum area of water container formed by two lines.

What interviewer tests
Two-pointer greedy: maximizing area by moving shorter pointer inward.

Approach (optimal)
Two pointers l=0, r=n-1. Compute area = min(height[l],height[r]) * (r-l). Move the pointer with smaller height inward (because moving the taller one cannot increase min height). Keep track of max.

Algorithm (step-by-step)

- l=0, r=n-1, maxArea=0
- while l < r:
    - area = Math.min(h[l],h[r])*(r-l)
    - update maxArea
    - if h[l] < h[r] l++ else r--

Java code

```
public int maxArea(int[] height) {
  int l = 0, r = height.length - 1;
  int max = 0;
  while (l < r) {
    int area = Math.min(height[l], height[r]) * (r - l);
    max = Math.max(max, area);
    if (height[l] < height[r]) l++;
    else r--;
  }
  return max;
```

}

Complexity

- Time: O(n)
- Space: O(1)

Example
[1,8,6,2,5,4,8,3,7] -> max area = 49 (left=1-index 2 height 8 and right last height 7).

---

8) Longest Common Prefix

Problem statement
Given an array of strings, find the longest common prefix among them. If there is none, return "".

What interviewer tests
String comparisons, edge/corner case handling. Multiple approaches: vertical scanning, horizontal scanning, divide-and-conquer, binary search.

Approach (simple & efficient)
Horizontal scanning: start with prefix = first string; for each next string, shrink prefix while it's not a prefix of that string (indexOf or startsWith).

Algorithm (step-by-step)

- If array empty return ""
- prefix = strs[0]
- For each i from 1..n-1:
    - while (!strs[i].startsWith(prefix)) prefix = prefix.substring(0, prefix.length()-1)
    - if prefix becomes empty return ""
- Return prefix

Java code

```
public String longestCommonPrefix(String[] strs) {
  if (strs == null || strs.length == 0) return "";
  String prefix = strs[0];
```

```
  for (int i = 1; i < strs.length; i++) {
    while (!strs[i].startsWith(prefix)) {
      prefix = prefix.substring(0, prefix.length() - 1);
      if (prefix.isEmpty()) return "";
    }
  }
  return prefix;
}
```

Complexity

- Time: O(S) where S is sum of all characters in the array (worst-case)
- Space: O(1) extra

Example
["flower","flow","flight"] -> "fl".

---

9) Group Anagrams

Problem statement
Given array of strings, group anagrams together. Return list of groups.

What interviewer tests
Hashing with canonical keys (sorted string or letter counts). Handling string transformations and grouping.

Approach (optimal)
Use Map<String, List<String>> where key is canonical form: either sorted characters of string or 26-count encoding (for lowercase a-z). For general strings, sorted key is easiest.

Algorithm (step-by-step)

- For each string s:
    - convert to char array, Arrays.sort(chars), key = new String(chars)
    - map.computeIfAbsent(key, k -> new ArrayList<>()).add(s)
- return new ArrayList<>(map.values())

Java code

```
public List<List<String>> groupAnagrams(String[] strs) {
   Map<String, List<String>> map = new HashMap<>();
   for (String s : strs) {
      char[] arr = s.toCharArray();
      Arrays.sort(arr);
      String key = new String(arr);
      map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
   }
   return new ArrayList<>(map.values());
}
```

Complexity

- Time: O(n * k log k) where k is average string length (sorting each string).
- Space: O(nk) for grouped output.

Optimization
If strings are only lowercase letters, using a 26-int count encoded as a string key gives O(n*k) time.

Example
["eat","tea","tan","ate","nat","bat"] -> groups like [["eat","tea","ate"],["tan","nat"],["bat"]].

---

10) Valid Parentheses

Problem statement
Given string containing characters '(', ')', '{', '}', '[' ,']', determine if input string is valid: brackets must close in correct order and type.

What interviewer tests
Stack usage and matching parentheses logic.

Approach (optimal)

Use a Stack. Push expected closing bracket when encountering opening bracket. When encountering a closing bracket, check if it matches top of stack.

Algorithm (step-by-step)

- If length odd -> invalid
- Create stack Deque<Character> st = new ArrayDeque<>()
- For each char c:
    - if c is '(', push ')'; '[' -> push ']' ; '{'-> push '}'
    - else if stack is empty or c != stack.pop() return false
- After loop, return stack.isEmpty()

Java code

```
public boolean isValid(String s) {
  Deque<Character> st = new ArrayDeque<>();
  for (char c : s.toCharArray()) {
    if (c == '(') st.push(')');
    else if (c == '[') st.push(']');
    else if (c == '{') st.push('}');
    else {
      if (st.isEmpty() || st.pop() != c) return false;
    }
  }
  return st.isEmpty();
}
```

Complexity

- Time: O(n)
- Space: O(n)

Example
"()[]{}" -> true. "(]" -> false.

---

11) Minimum Window Substring

Problem statement
Given strings s and t, return the minimum window in s which will contain all characters in t. If no such window exists, return "".

What interviewer tests
Complex sliding-window with frequency counters, careful contraction and expansion loops.

Approach (optimal)
Use sliding window with two maps/arrays: one for required counts (for t) and one for current window counts. Expand right pointer to satisfy requirement, then contract left pointer to minimize window while still valid. Maintain formed count to know when window contains all required characters.

Algorithm (step-by-step)

- Build Map<Character, Integer> need for t counts.
- window map counts current window.
- required = need.size() number of unique chars required.
- formed = 0 counts how many unique chars match required count in window.
- l = 0, r = 0, minLen = INF, minL = 0
- while r < s.length:
    - add s[r] to window; if window.get(c).equals(need.get(c)) formed++
    - while l <= r and formed == required:
        - try update min window
        - remove s[l] from window; if window.get(c) < need.get(c) formed-- ; l++
    - r++
- return min substring if found

Java code

```
public String minWindow(String s, String t) {
    if (s == null || t == null || s.length() < t.length()) return "";
    Map<Character, Integer> need = new HashMap<>();
    for (char c : t.toCharArray()) need.put(c, need.getOrDefault(c, 0) + 1);

    Map<Character, Integer> window = new HashMap<>();
    int required = need.size();
    int formed = 0;
    int l = 0, r = 0;
    int[] ans = {-1, 0, 0}; // length, left, right

    while (r < s.length()) {
        char c = s.charAt(r);
        window.put(c, window.getOrDefault(c, 0) + 1);

        if (need.containsKey(c) && window.get(c).intValue() == need.get(c).intValue()) {
            formed++;
        }

        while (l <= r && formed == required) {
            c = s.charAt(l);
            // update answer
            if (ans[0] == -1 || r - l + 1 < ans[0]) {
                ans[0] = r - l + 1;
                ans[1] = l;
                ans[2] = r;
```

```
        }

        window.put(c, window.get(c) - 1);
        if (need.containsKey(c) && window.get(c).intValue() < need.get(c).intValue()) {
          formed--;
        }
        l++;
      }
      r++;
    }
    return ans[0] == -1 ? "" : s.substring(ans[1], ans[2] + 1);
}
```

Complexity

- Time: O(|s| + |t|) in practice; each char visited at most twice.
- Space: O(|s| + |t|) for maps but practically O(charset).

Example
s = "ADOBECODEBANC", t = "ABC" -> output "BANC".

---

12) Merge Intervals

Problem statement
Given array of intervals [[s1,e1], [s2,e2], ...], merge all overlapping intervals and return array of merged intervals.

What interviewer tests
Sorting + merging intervals, edge case handling for contiguous/overlap boundaries.

Approach (optimal)
Sort intervals by start. Iterate and merge with last interval in result: if current.start <= last.end -> merge last.end = max(last.end, current.end) else append new interval.

Algorithm (step-by-step)

- Sort intervals by start ascending.
- Initialize List<int[]> res = new ArrayList<>().
- For each interval cur:
    - if res empty or res.last.end < cur.start -> add cur
    - else update res.last.end = max(res.last.end, cur.end)
- Return result converted to array.

Java code

```
public int[][] merge(int[][] intervals) {
  if (intervals.length <= 1) return intervals;
  Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
  List<int[]> res = new ArrayList<>();
  int[] current = intervals[0];
  res.add(current);
  for (int[] interval : intervals) {
    int curStart = interval[0], curEnd = interval[1];
    if (current[1] < curStart) {
      current = interval;
      res.add(current);
    } else {
      current[1] = Math.max(current[1], curEnd);
    }
  }
  return res.toArray(new int[res.size()][]);
}
```

Complexity

- Time: O(n log n) due to sorting.

- Space: O(n) for output.

Example
[[1,3],[2,6],[8,10],[15,18]] -> [[1,6],[8,10],[15,18]].

13) Two Sum (Hashmap Variant)

(Covered earlier but rewritten for Hashing category completeness)

Problem

Return indices of two numbers in nums such that their sum equals target. Exactly one solution exists.

What's being tested?

- Hash table lookups
- O(n) one-pass strategy
- Key → index mapping

Optimal Approach

Use a HashMap storing each number's index. For each number, check if its complement already exists.

Algorithm

1. Create empty map value -> index
2. For each index i:
    - complement = target - nums[i]
    - if complement exists in map → return [map.get(complement), i]
    - else store nums[i] → i

Java Code

```
public int[] twoSum(int[] nums, int target) {
  Map<Integer, Integer> map = new HashMap<>();
  for (int i = 0; i < nums.length; i++) {
    int complement = target - nums[i];
    if (map.containsKey(complement)) {
```

```
            return new int[]{map.get(complement), i};
        }
        map.put(nums[i], i);
    }
    return new int[]{};
}
```

Complexity

- Time: **O(n)**
- Space: **O(n)**

Example

nums = [2,7,11,15], target=9 → return [0,1].

---

14) Top K Frequent Elements

Problem

Given an array nums, return the k most frequent elements.

What's being tested?

- HashMap frequency counting
- Min-heap or bucket sort
- Understanding of time complexity tradeoffs

Optimal Approach (Heap)

Use a HashMap to store frequency, then a min-heap of size k.

Algorithm

1. Count frequencies in Map<Integer, Integer> freq.
2. Maintain a min-heap (value, frequency) of size k.
3. Add elements to heap:
    - If heap size > k → poll

4. Extract values from heap.

Java Code

```
public int[] topKFrequent(int[] nums, int k) {
   Map<Integer, Integer> freq = new HashMap<>();
   for (int n : nums) freq.put(n, freq.getOrDefault(n, 0) + 1);

   PriorityQueue<int[]> heap = new PriorityQueue<>((a, b) -> a[1] - b[1]);

   for (int n : freq.keySet()) {
      heap.offer(new int[]{n, freq.get(n)});
      if (heap.size() > k) heap.poll();
   }

   int[] result = new int[k];
   for (int i = k - 1; i >= 0; i--) {
      result[i] = heap.poll()[0];
   }
   return result;
}
```

Complexity

- Time: **O(n log k)**
- Space: **O(n)**

Example

nums = [1,1,1,2,2,3], k=2 → [1,2].

---

15) Longest Consecutive Sequence

Problem

Given array nums, return the length of the longest consecutive elements sequence. Must run in **O(n)**.

What's being tested?

- HashSet lookups
- O(n) reasoning
- Detecting sequence "starts"

Optimal Approach (HashSet)

A number begins a sequence only if num - 1 is not present. Expand forward: num + 1, num + 2, ...

Algorithm

1. Insert all numbers into a HashSet.
2. For each number:
    - If num - 1 not in set → start new sequence
    - Expand: count length until numbers stop
3. Track max length.

Java Code

```
public int longestConsecutive(int[] nums) {
  Set<Integer> set = new HashSet<>();
  for (int n : nums) set.add(n);

  int longest = 0;

  for (int n : set) {
    if (!set.contains(n - 1)) { // start of sequence
      int curr = n;
      int length = 1;
```

```
        while (set.contains(curr + 1)) {
          curr++;
          length++;
        }
        longest = Math.max(longest, length);
      }
    }
    return longest;
}
```

Complexity

- Time: **O(n)** (every number visited once)
- Space: **O(n)**

Example

[100,4,200,1,3,2] → longest consecutive sequence is [1,2,3,4] → answer = 4.

---

16) Subarray Sum Equals K

Problem

Given an integer array nums and integer k, find the number of subarrays whose sum equals k.

What's being tested?

- Prefix sum + HashMap
- Pattern for subarray-sum problems
- Recognition that subarray sum = prefix[j] - prefix[i]

Optimal Approach (Prefix-Sum HashMap)

If prefixSum[j] - prefixSum[i] = k, then a subarray (i+1..j) equals k.
Track the count of each prefixSum.

Algorithm

1. Map stores: prefixSum → frequency.
2. Initialize with {0:1} (empty prefix).
3. Running sum: sum += nums[i].
4. Check if (sum - k) exists → add its freq to result.
5. Store sum in map.

Java Code

```
public int subarraySum(int[] nums, int k) {
   Map<Integer, Integer> map = new HashMap<>();
   map.put(0, 1);
   int sum = 0, count = 0;


   for (int n : nums) {
      sum += n;
      if (map.containsKey(sum - k)) {
         count += map.get(sum - k);
      }
      map.put(sum, map.getOrDefault(sum, 0) + 1);
   }
   return count;
}
```

Complexity

- Time: **O(n)**
- Space: **O(n)**

Example

nums = [1,1,1], k = 2 → subarrays: [1,1] twice → result = 2.

---

17) Happy Number

Problem

A number is happy if repeatedly replacing it by sum of squares of digits eventually becomes 1.
If it loops endlessly in a cycle → not happy.

What's being tested?

- HashSet to detect cycles
- Math digit operations
- Loop detection

Approach (HashSet Cycle Detection)

Compute next number; if you see a number already seen → cycle → false.

Algorithm

1. Create a HashSet.
2. While n != 1 and n not in set:
    - add n
    - compute next = sum of squares of digits
3. return n == 1.

Java Code

```
public boolean isHappy(int n) {
  Set<Integer> seen = new HashSet<>();
  while (n != 1 && !seen.contains(n)) {
    seen.add(n);
    n = sumOfSquares(n);
  }
  return n == 1;
}
```

```
private int sumOfSquares(int n) {
   int sum = 0;
   while (n > 0) {
      int digit = n % 10;
      sum += digit * digit;
      n /= 10;
   }
   return sum;
}
```

Complexity

- Time: O(log n * cycle length)
- Space: O(cycle size)

Example:
19 → 1 so it is happy.

---

18) LRU Cache

Problem

Design a data structure that supports:

- get(key)
- put(key, value)
  All in **O(1)** time. When capacity exceeded → remove least recently used entry.

What's being tested?

- HashMap + Doubly Linked List combination
- Ability to design data structures
- Understanding O(1) insert/delete in a list
- System design thinking at small scale

Optimal Approach

Use:

- HashMap → key to Node
- Doubly Linked List → tracks usage (most recent at right end)
  Operations:
- get: move node to end
- put: insert/update node; if capacity reached → remove head

Diagram (mental model)

HEAD (LRU) ↔ ... ↔ TAIL (MRU)

Algorithm

1. For get(key):
    - If not in map → return -1
    - Move node to tail
    - Return value
2. For put(key, value):
    - If exists → update value, move to tail
    - Else:
        - Create new node, add to tail
        - Add key in map
        - If over capacity → remove head node and its key

Java Code (clean & interview-ready)

```
class LRUCache {
  class Node {
    int key, value;
    Node prev, next;
    Node(int k, int v) { key = k; value = v; }
```

```
}

private final int capacity;
private Map<Integer, Node> map;
private Node head, tail;

public LRUCache(int capacity) {
    this.capacity = capacity;
    map = new HashMap<>();
    head = new Node(0, 0);
    tail = new Node(0, 0);
    head.next = tail;
    tail.prev = head;
}

public int get(int key) {
    if (!map.containsKey(key)) return -1;
    Node node = map.get(key);
    remove(node);
    insert(node);
    return node.value;
}

public void put(int key, int value) {
    if (map.containsKey(key)) {
        remove(map.get(key));
```

```
        }
        Node node = new Node(key, value);
        insert(node);
        map.put(key, node);

        if (map.size() > capacity) {
            Node lru = head.next;
            remove(lru);
            map.remove(lru.key);
        }
    }

    private void remove(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void insert(Node node) {
        node.prev = tail.prev;
        node.next = tail;
        tail.prev.next = node;
        tail.prev = node;
    }
}
```

HEAD <-> node1 <-> node2 <-> node3 <

HEAD <-> node1 <-> node3 <->

HEAD <-> node1 <-> node3 <-> nodeX <

Complexity

- Time: **O(1)** for both get & put

- Space: **O(capacity)**

Example

LRUCache cache = new LRUCache(2)

cache.put(1,1)

cache.put(2,2)

cache.get(1) → returns 1 (1 becomes MRU)

cache.put(3,3) → evicts key 2

✅ **LINKED LISTS — DETAILED Q&A (6 Problems)**

---

19) Reverse Linked List

Problem

Given the head of a singly linked list, reverse the list and return the new head.

What's being tested?

- Pointer manipulation
- Understanding of in-place updates
- Iterative vs recursive solutions

Approach (Optimal: Iterative)

Keep 3 pointers: prev, curr, next.
Reverse curr.next = prev as you traverse.

Algorithm

1. Initialize prev = null, curr = head.
2. While curr != null
    - next = curr.next
    - curr.next = prev
    - move prev = curr
    - move curr = next

3. Return prev (new head)

Java Code

```
public ListNode reverseList(ListNode head) {
  ListNode prev = null, curr = head;
  while (curr != null) {
    ListNode next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
  }
  return prev;
}
```

Complexity

- Time: O(n)
- Space: O(1)

Example

1 → 2 → 3 → 4 → reversed → 4 → 3 → 2 → 1

---

20) Merge Two Sorted Lists

Problem

Given two sorted linked lists, merge them into one sorted list.

What's being tested?

- Linked-list construction
- Dummy node technique
- Handling null lists gracefully

Approach (Optimal)

Use a dummy head and pointer. Compare nodes from both lists and append smaller one.

Algorithm

1. Create dummy node, tail = dummy.
2. While both lists not empty:
    - append smaller node
    - move pointer
3. Append remaining nodes
4. Return dummy.next

Java Code

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
  ListNode dummy = new ListNode(0);
  ListNode tail = dummy;


  while (l1 != null && l2 != null) {
    if (l1.val < l2.val) {
      tail.next = l1;
      l1 = l1.next;
    } else {
      tail.next = l2;
      l2 = l2.next;
    }
    tail = tail.next;
  }
  tail.next = (l1 != null) ? l1 : l2;
  return dummy.next;
}
```

Complexity

- Time: O(m+n)
- Space: O(1)

Example

1→2→4 and 1→3→4 → 1,1,2,3,4,4

---

21) Add Two Numbers (Linked List)

Problem

Given two linked lists representing two non-negative integers (digits stored in reverse order), return their sum as a linked list.

What's being tested?

- Linked-list traversal
- Carry manipulation
- Handling different lengths

Approach

Simulate addition digit-by-digit, maintain carry, create result list using dummy node.

Algorithm

1. Initialize carry = 0
2. Traverse both lists until both null and carry = 0
3. Sum = l1.val + l2.val + carry
4. node.val = sum % 10
5. carry = sum / 10
6. Move pointers

Java Code

```
public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
   ListNode dummy = new ListNode(0);
```

```
    ListNode tail = dummy;
    int carry = 0;

    while (l1 != null || l2 != null || carry > 0) {
        int sum = carry;
        if (l1 != null) { sum += l1.val; l1 = l1.next; }
        if (l2 != null) { sum += l2.val; l2 = l2.next; }

        tail.next = new ListNode(sum % 10);
        tail = tail.next;
        carry = sum / 10;
    }

    return dummy.next;
}
```

Complexity

- Time: O(max(m, n))
- Space: O(max(m, n)) output list

Example

l1 = 2→4→3 (342)

l2 = 5→6→4 (465)

sum = 807 → 7→0→8

---

22) Linked List Cycle Detection

Problem

Detect if a linked list contains a cycle.

What's being tested?

- Floyd's Tortoise and Hare algorithm
- Understanding of fast/slow pointers

Approach (Optimal)

Use slow pointer moving 1 step, fast pointer moving 2 steps.
If they ever meet → cycle exists.

Algorithm

1. If head null → false
2. slow = head, fast = head.next
3. While fast != null and fast.next != null:
    - if slow == fast → true
    - move slow by 1, fast by 2
4. Return false

Java Code

```
public boolean hasCycle(ListNode head) {
  if (head == null || head.next == null) return false;
  ListNode slow = head, fast = head.next;

  while (fast != null && fast.next != null) {
    if (slow == fast) return true;
    slow = slow.next;
    fast = fast.next.next;
  }
  return false;
}
```

Complexity

- Time: O(n)
- Space: O(1)

Example

Cycle at node 2 → fast/slow meet.

---

23) Remove Nth Node From End

Problem

Remove the Nth node from the end of a linked list in one pass.

What's being tested?

- Two-pointer technique
- Handling removal of head
- Edge cases

Approach (Two-pointers)

Use fast pointer first move N steps.
Then move both until fast reaches end → slow points to node before one to remove.

Algorithm

1. Dummy → head
2. fast = slow = dummy
3. Move fast N+1 steps
4. Move fast & slow until fast null
5. Remove slow.next
6. Return dummy.next

Java Code

```
public ListNode removeNthFromEnd(ListNode head, int n) {
  ListNode dummy = new ListNode(0);
  dummy.next = head;
```

```
    ListNode fast = dummy, slow = dummy;

    for (int i = 0; i <= n; i++) {
        fast = fast.next;
    }

    while (fast != null) {
        fast = fast.next;
        slow = slow.next;
    }

    slow.next = slow.next.next;
    return dummy.next;
}
```

Complexity

- Time: O(n)
- Space: O(1)

Example

1→2→3→4→5, n=2 → remove 4 → output 1→2→3→5

---

24) Copy List with Random Pointer

Problem

Each node has next and random pointer.
Return a **deep copy** of the list.

What's being tested?

- HashMap for old→new mapping OR

- Interleaving trick (optimal, no extra map)
- Deep vs shallow copying

Optimal Approach (Interleaving Trick)

Three steps:

1. **Interleave copy nodes**:
   A → B → C becomes
   A → A' → B → B' → C → C'
2. **Assign random pointers**:
   A'.random = A.random.next
3. **Unweave lists**:
   Separate original and copied list.

Algorithm

1. Copy nodes & weave them
2. Set random pointers
3. Unweave

Java Code

```
public Node copyRandomList(Node head) {
  if (head == null) return null;


  Node curr = head;
  // Step 1: Interleave copy nodes
  while (curr != null) {
    Node copy = new Node(curr.val);
    copy.next = curr.next;
    curr.next = copy;
    curr = copy.next;
  }
```

```
    // Step 2: Assign random pointers
    curr = head;
    while (curr != null) {
        if (curr.random != null) {
            curr.next.random = curr.random.next;
        }
        curr = curr.next.next;
    }

    // Step 3: Unweave the two lists
    curr = head;
    Node copyHead = head.next;
    while (curr != null) {
        Node copy = curr.next;
        curr.next = copy.next;
        if (copy.next != null) copy.next = copy.next.next;
        curr = curr.next;
    }

    return copyHead;
}
```

Complexity

- Time: **O(n)**
- Space: **O(1)** (no HashMap used)

25) Binary Tree Traversals (Preorder / Inorder / Postorder)

Problem
Given the root of a binary tree, return the preorder (root-left-right), inorder (left-root-right), and postorder (left-right-root) traversals. Implement recursive and iterative approaches.

What interviewer tests
Recursion fluency, stack usage for iterative traversal, understanding traversal orders and use-cases (e.g., inorder of BST gives sorted order).

Approaches & Intuition

- Recursive: natural and concise.
- Iterative: use a stack. Iterative inorder uses stack to simulate recursion; iterative preorder can push right then left to stack; iterative postorder is trickier (use two stacks or modify preorder).

Algorithms (high-level)

- Preorder recursion: visit node, recurse left, recurse right.
- Inorder recursion: recurse left, visit node, recurse right.
- Postorder recursion: recurse left, recurse right, visit node.
- Iterative inorder: push left children to stack until null; pop, visit, go right.

Java (recursive + iterative inorder example)

```
// Tree node class
class TreeNode {
  int val;
  TreeNode left, right;
  TreeNode(int v) { val = v; }
}


// Recursive traversals (examples)
void preorder(TreeNode node, List<Integer> out) {
  if (node == null) return;
```

```
    out.add(node.val);
    preorder(node.left, out);
    preorder(node.right, out);
}
void inorder(TreeNode node, List<Integer> out) {
    if (node == null) return;
    inorder(node.left, out);
    out.add(node.val);
    inorder(node.right, out);
}
void postorder(TreeNode node, List<Integer> out) {
    if (node == null) return;
    postorder(node.left, out);
    postorder(node.right, out);
    out.add(node.val);
}

// Iterative inorder
List<Integer> inorderIterative(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;
    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
```

```
        }
        curr = stack.pop();
        res.add(curr.val);
        curr = curr.right;
    }
    return res;
}
```

Complexity

- Time: O(n) for all traversals.
- Space: O(h) recursion/stack, where h is tree height (O(n) worst-case).

Example
Tree:

```
  1
 / \
2   3
/ \
4  5
```

- Preorder: [1,2,4,5,3]
- Inorder: [4,2,5,1,3]
- Postorder:[4,5,2,3,1]

---

26) Validate Binary Search Tree (BST)

Problem
Given a binary tree, determine if it is a valid BST: for every node, all values in left subtree < node.val < all values in right subtree.

What interviewer tests
Understanding of BST invariants and how to enforce them across subtrees.

Approaches & Intuition

- Inorder traversal should be strictly increasing for valid BST (simple).
- Or recursively pass down allowable min/max bounds for each node.

Algorithm (min/max bounds)

- Recursively check: node.val must be > min and < max. For left child update max=node.val; for right child update min=node.val.

Java

```
boolean isValidBST(TreeNode root) {
  return isValidBST(root, null, null);
}
boolean isValidBST(TreeNode node, Integer lower, Integer upper) {
  if (node == null) return true;
  int val = node.val;
  if (lower != null && val <= lower) return false;
  if (upper != null && val >= upper) return false;
  if (!isValidBST(node.left, lower, val)) return false;
  if (!isValidBST(node.right, val, upper)) return false;
  return true;
}
```

Complexity

- Time: O(n)
- Space: O(h) recursion stack

Example

Tree:

```
 2
/ \
1  3
```

Valid BST → true. If right child had value 2 → invalid.

---

27) Level Order Traversal (BFS)

Problem
Return level order (breadth-first) traversal of a binary tree as list of levels.

What interviewer tests
BFS using a queue, handling level boundaries, typical in serialization & tree breadth queries.

Approach & Algorithm
Use a queue; for each level, process size = queue.size() nodes, add their children to queue, collect level values.

Java

```
List<List<Integer>> levelOrder(TreeNode root) {
  List<List<Integer>> res = new ArrayList<>();
  if (root == null) return res;
  Deque<TreeNode> q = new ArrayDeque<>();
  q.add(root);
  while (!q.isEmpty()) {
    int size = q.size();
    List<Integer> level = new ArrayList<>();
    for (int i = 0; i < size; i++) {
      TreeNode node = q.poll();
      level.add(node.val);
      if (node.left != null) q.add(node.left);
      if (node.right != null) q.add(node.right);
    }
    res.add(level);
```

```
    }
    return res;
}
```

Complexity

- Time: O(n)
- Space: O(width) (max number of nodes at any level)

Example
Input same as previous tree → [[1],[2,3],[4,5]].

---

28) Lowest Common Ancestor (LCA) of BST

Problem
Given a BST and two node values p and q, find their lowest common ancestor (node with largest depth that is ancestor of both).

What interviewer tests
Leveraging BST properties for efficient search (no need to traverse whole tree).

Approach & Intuition

- If both p and q less than node.val → LCA in left subtree.
- If both greater → right subtree.
- Else current node is LCA.

Algorithm (iterative or recursive)
Traverse from root: if p.val < root.val and q.val < root.val → root = root.left; else if both > → root = root.right; else return root.

Java

```
TreeNode lowestCommonAncestorBST(TreeNode root, TreeNode p, TreeNode q) {
    TreeNode curr = root;
    while (curr != null) {
        if (p.val < curr.val && q.val < curr.val) curr = curr.left;
        else if (p.val > curr.val && q.val > curr.val) curr = curr.right;
```

```
    else return curr;
  }
  return null;
}
```

Complexity

- Time: O(h) where h is tree height
- Space: O(1)

Example
BST:

```
   6
  / \
 2   8
/ \ / \
0 4 7 9
```

LCA of 2 and 8 = 6. LCA of 2 and 4 = 2.

---

29) Serialize and Deserialize Binary Tree

Problem
Design methods to serialize a binary tree to a string and deserialize back to the tree structure.

What interviewer tests
Tree encoding/decoding, BFS/DFS serialization choices, handling nulls and unique delimiters, correctness under all shapes.

Approaches & Intuition

- BFS (level-order) with placeholders (e.g., "null") produces compact representation and is easy to parse.
- DFS (preorder) with null markers also works.

Algorithm (BFS serialization example)

- Serialize: BFS queue, append node values or "null" for absent children separated by commas. Trim trailing nulls for compactness.
- Deserialize: split tokens, create root from first token, use queue to assign children sequentially.

Java

```
// Serialization (level order)
String serialize(TreeNode root) {
  if (root == null) return "";
  StringBuilder sb = new StringBuilder();
  Deque<TreeNode> q = new ArrayDeque<>();
  q.add(root);
  while (!q.isEmpty()) {
    TreeNode node = q.poll();
    if (node == null) {
      sb.append("null,");
      continue;
    }
    sb.append(node.val).append(",");
    q.add(node.left);
    q.add(node.right);
  }
  // optional: trim trailing "null," entries
  return sb.toString();
}

// Deserialization
TreeNode deserialize(String data) {
```

```
    if (data == null || data.isEmpty()) return null;
    String[] tokens = data.split(",");
    TreeNode root = new TreeNode(Integer.parseInt(tokens[0]));
    Deque<TreeNode> q = new ArrayDeque<>();
    q.add(root);
    int i = 1;
    while (!q.isEmpty() && i < tokens.length) {
        TreeNode node = q.poll();
        String leftTok = tokens[i++];
        if (!leftTok.equals("null")) {
            node.left = new TreeNode(Integer.parseInt(leftTok));
            q.add(node.left);
        }
        if (i < tokens.length) {
            String rightTok = tokens[i++];
            if (!rightTok.equals("null")) {
                node.right = new TreeNode(Integer.parseInt(rightTok));
                q.add(node.right);
            }
        }
    }
    return root;
}
```

Complexity

- Time: O(n) both serialize & deserialize.
- Space: O(n) for output and queue.

Example
Tree 1,2,3,null,null,4,5 serialized → "1,2,3,null,null,4,5,null,null,null,null," (format may vary).

---

30) Diameter of Binary Tree

Problem
Given a binary tree, return the length (number of edges) of the diameter — the longest path between any two nodes.

What interviewer tests
Tree DP (postorder), combining child results into global metric, careful handling of heights.

Approach & Intuition
For every node, the longest path through that node equals leftHeight + rightHeight.
Compute heights recursively and update global max diameter.

Algorithm

- Postorder traversal returns height for node = 1 + max(leftHeight, rightHeight).
- At each node update maxDiameter = max(maxDiameter, leftHeight + rightHeight).

Java

```
int maxDiameter = 0;

int diameterOfBinaryTree(TreeNode root) {
  dfsHeight(root);
  return maxDiameter;
}

int dfsHeight(TreeNode node) {
  if (node == null) return 0;
  int lh = dfsHeight(node.left);
  int rh = dfsHeight(node.right);
  maxDiameter = Math.max(maxDiameter, lh + rh);
  return 1 + Math.max(lh, rh);
```

}

Complexity

- Time: O(n) (each node visited once)
- Space: O(h) recursion stack

Example
Tree:

1

/ \

2 3

/ \

4 5

Longest path: 4→2→1→3 length (edges) = 3.

---

31) Kth Smallest Element in BST

Problem
Given a BST, find the k-th smallest element (1-indexed).

What interviewer tests
Inorder traversal yields sorted order; ability to do iterative inorder to achieve O(h + k) time without storing all values.

Approach & Intuition
Do iterative inorder using a stack; pop nodes until we've visited k nodes.

Algorithm

- Use stack, push left children down to smallest.
- While stack not empty:
    - pop node, decrement k; if k==0 return node.val
    - move to node.right and push its left spine.

Java

```
int kthSmallest(TreeNode root, int k) {
  Deque<TreeNode> stack = new ArrayDeque<>();
  TreeNode curr = root;
  while (curr != null || !stack.isEmpty()) {
    while (curr != null) {
      stack.push(curr);
      curr = curr.left;
    }
    curr = stack.pop();
    if (--k == 0) return curr.val;
    curr = curr.right;
  }
  throw new IllegalArgumentException("k too large");
}
```

Complexity

- Time: O(h + k) where h is tree height
- Space: O(h) stack

Example
BST inorder: [1,2,3,4,5]; k=3 → returns 3.

---

32) Binary Tree Maximum Path Sum

Problem
Given a non-empty binary tree, find the maximum path sum. Path may start and end at any node (must contain at least one node).

What interviewer tests
Similar to diameter but with sums and negative values — ability to ignore negative contributions and combine children optimally.

Approach & Intuition

At node, compute maxGain(node) = node.val + max(0, leftGain, rightGain) where leftGain/rightGain are max contributions from children. Update global maxSum = max(maxSum, node.val + max(0,leftGain) + max(0,rightGain)).

Algorithm

- Postorder: compute leftGain, rightGain
- localMax = node.val + max(0,leftGain) + max(0,rightGain) → update global max
- return node.val + max(0,leftGain, rightGain) to parent

Java

```
int maxSum = Integer.MIN_VALUE;

int maxPathSum(TreeNode root) {
   maxGain(root);
   return maxSum;
}

int maxGain(TreeNode node) {
   if (node == null) return 0;
   int left = Math.max(0, maxGain(node.left));
   int right = Math.max(0, maxGain(node.right));
   maxSum = Math.max(maxSum, node.val + left + right);
   return node.val + Math.max(left, right);
}
```

Complexity

- Time: O(n)
- Space: O(h) recursion stack

Example

```
  -10
  / \
```

9 20

/ \

15 7

Max path sum = 15 + 20 + 7 = 42.

33) Number of Islands

Problem
Given a 2D grid of '1' (land) and '0' (water), return the number of islands. An island is connected horizontally or vertically.

What interviewer tests
Graph traversal (BFS/DFS), marking visited nodes, component counting.

Approach (DFS or BFS)
Treat grid cells as graph nodes. For each unvisited land cell, run DFS/BFS to mark the entire connected component, increment count.

Algorithm (DFS)

1. Iterate all cells.
2. When you find '1', increment count and call dfs(i,j) which:
    - If out of bounds or cell != '1' return.
    - Mark cell as visited (set to '0' or use boolean visited).
    - Recurse to 4 neighbors.
3. Continue.

Java

```
public int numIslands(char[][] grid) {
  if (grid == null || grid.length == 0) return 0;
  int m = grid.length, n = grid[0].length;
  int count = 0;
  for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
```

```
        if (grid[i][j] == '1') {
            count++;
            dfs(grid, i, j, m, n);
        }
      }
    }
    return count;
}

private void dfs(char[][] grid, int i, int j, int m, int n) {
    if (i < 0 || j < 0 || i >= m || j >= n || grid[i][j] != '1') return;
    grid[i][j] = '0'; // mark visited
    dfs(grid, i+1, j, m, n);
    dfs(grid, i-1, j, m, n);
    dfs(grid, i, j+1, m, n);
    dfs(grid, i, j-1, m, n);
}
```

Complexity

- Time: O(m * n) — each cell visited once.
- Space: O(m * n) worst-case recursion stack (island fills grid) or O(min(m,n)) average for recursion depth.

Example
Grid:

11110

11010

11000

00000

→ 1 island.

---

34) Clone Graph

Problem
Given a reference of a node in a connected undirected graph, return a deep copy (clone) of the graph. Each node contains val and neighbors.

What interviewer tests
Graph traversal (BFS/DFS) with mapping original → copy to avoid revisiting; understanding of deep copy semantics.

Approach (BFS or DFS with HashMap)
Use a HashMap orig -> copy. Traverse graph and create copies on the fly; set neighbor references using map.

Algorithm (BFS)

1. If node == null return null.
2. Create map and queue; put node -> new Node(node.val); enqueue node.
3. While queue not empty:
    - pop cur
    - for each neighbor nei:
        - if not in map → create copy and enqueue nei
        - add map.get(nei) to map.get(cur).neighbors
4. Return map.get(node)

Java

```
class Node {
  public int val;
  public List<Node> neighbors;
  public Node(int v) { val = v; neighbors = new ArrayList<>(); }
}
```

```
public Node cloneGraph(Node node) {
  if (node == null) return null;
  Map<Node, Node> map = new HashMap<>();
  Queue<Node> q = new ArrayDeque<>();
  map.put(node, new Node(node.val));
  q.add(node);
  while (!q.isEmpty()) {
    Node cur = q.poll();
    for (Node nei : cur.neighbors) {
      if (!map.containsKey(nei)) {
        map.put(nei, new Node(nei.val));
        q.add(nei);
      }
      map.get(cur).neighbors.add(map.get(nei));
    }
  }
  return map.get(node);
}
```

Complexity

- Time: O(V + E) — each node and edge processed.
- Space: O(V) for map + queue.

Example
A triangle graph 1–2–3–1 → cloned triangle with distinct node objects.

---

35) Course Schedule (Topological Sort / Cycle Detection)

Problem
Given numCourses and prerequisites pairs [a,b] meaning to take course a you must first take b, return whether you can finish all courses (i.e., graph is acyclic).

What interviewer tests
Topological sort (Kahn's algorithm) or DFS-based cycle detection. Understanding of DAGs.

Approach (Kahn's BFS)
Compute indegree for all nodes. Start with nodes of indegree 0, repeatedly remove and decrement indegrees. If processed nodes == numCourses → no cycle.

Algorithm (Kahn)

1. Build adjacency list and indegree array.
2. Enqueue all nodes with indegree 0.
3. Count popped nodes; for each popped node, decrement indegree of neighbors and enqueue if becomes 0.
4. Return count == numCourses.

Java

```
public boolean canFinish(int numCourses, int[][] prerequisites) {
  List<List<Integer>> adj = new ArrayList<>();
  for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
  int[] indeg = new int[numCourses];
  for (int[] p : prerequisites) {
    int a = p[0], b = p[1];
    adj.get(b).add(a);
    indeg[a]++;
  }
  Queue<Integer> q = new ArrayDeque<>();
  for (int i = 0; i < numCourses; i++) if (indeg[i] == 0) q.add(i);
  int count = 0;
  while (!q.isEmpty()) {
```

```
      int u = q.poll();
      count++;
      for (int v : adj.get(u)) {
        indeg[v]--;
        if (indeg[v] == 0) q.add(v);
      }
    }
    return count == numCourses;
}
```

Complexity

- Time: O(V + E)
- Space: O(V + E)

Example
numCourses=2, prerequisites=[[1,0]] → true. If prerequisites=[[1,0],[0,1]] → false (cycle).

---

36) Minimum Height Trees

Problem
For an undirected tree with n nodes labeled 0..n-1 and edges list, return all root labels that produce minimum height trees (MHTs).

What interviewer tests
Graph trimming / topological layers technique — remove leaves iteratively until 1 or 2 centers remain.

Approach (Peeling leaves)
Leaves are nodes with degree 1. Remove leaves layer by layer; remaining 1 or 2 nodes are centers (MHT roots).

Algorithm

1. Handle small n (<=2) return all nodes.
2. Build adjacency list and degree array.

3. Initialize queue with all leaves.
4. While remaining nodes > 2:
    - remove all current leaves (size times), decrement neighbor degrees; if neighbor becomes leaf add to queue.
5. Remaining nodes in queue are centers.

Java

```
public List<Integer> findMinHeightTrees(int n, int[][] edges) {
  if (n <= 0) return new ArrayList<>();
  if (n == 1) return Collections.singletonList(0);
  List<Set<Integer>> adj = new ArrayList<>();
  for (int i = 0; i < n; i++) adj.add(new HashSet<>());
  for (int[] e : edges) {
    adj.get(e[0]).add(e[1]);
    adj.get(e[1]).add(e[0]);
  }
  List<Integer> leaves = new ArrayList<>();
  for (int i = 0; i < n; i++) if (adj.get(i).size() == 1) leaves.add(i);

  int remaining = n;
  while (remaining > 2) {
    remaining -= leaves.size();
    List<Integer> newLeaves = new ArrayList<>();
    for (int leaf : leaves) {
      int neighbor = adj.get(leaf).iterator().next();
      adj.get(neighbor).remove(leaf);
      if (adj.get(neighbor).size() == 1) newLeaves.add(neighbor);
```

```
        }
        leaves = newLeaves;
    }
    return leaves;
}
```

Complexity

- Time: O(n) — each edge/node processed constant times.
- Space: O(n)

Example
n=4 edges=[[1,0],[1,2],[1,3]] → center = [1].

---

37) Word Ladder

Problem
Given beginWord, endWord, and a wordList, return the length of the shortest transformation sequence from begin to end where only one letter can be changed at a time and each intermediate word must exist in wordList.

What interviewer tests
BFS on implicit graph (each word is a node); performance optimization using intermediate generic states or bidirectional BFS.

Approach (BFS with generic states)
Precompute mapping from intermediate patterns with a * placeholder (e.g., h*t) to list of words. Use BFS from beginWord to endWord.

Algorithm

1. If end not in wordList return 0.
2. Preprocess wordList: for each word and each position, create key word.substring(0,i) + '*' + word.substring(i+1) → add word to map.
3. BFS starting from beginWord, for each word generate all intermediate keys, iterate neighbor words, mark visited, enqueue with depth+1. Stop when reach endWord.

Java

```
public int ladderLength(String beginWord, String endWord, List<String> wordList) {
  Set<String> wordSet = new HashSet<>(wordList);
  if (!wordSet.contains(endWord)) return 0;

  int L = beginWord.length();
  Map<String, List<String>> allCombo = new HashMap<>();
  for (String word : wordSet) {
    for (int i = 0; i < L; i++) {
      String key = word.substring(0, i) + "*" + word.substring(i + 1);
      allCombo.computeIfAbsent(key, k -> new ArrayList<>()).add(word);
    }
  }

  Queue<Pair<String,Integer>> q = new ArrayDeque<>();
  q.add(new Pair<>(beginWord, 1));
  Set<String> visited = new HashSet<>();
  visited.add(beginWord);

  while (!q.isEmpty()) {
    Pair<String,Integer> p = q.poll();
    String word = p.getKey();
    int level = p.getValue();
    for (int i = 0; i < L; i++) {
      String key = word.substring(0, i) + "*" + word.substring(i + 1);
      List<String> neighbors = allCombo.getOrDefault(key, Collections.emptyList());
      for (String nei : neighbors) {
```

```
            if (nei.equals(endWord)) return level + 1;
            if (!visited.contains(nei)) {
              visited.add(nei);
              q.add(new Pair<>(nei, level + 1));
            }
          }
          // optional optimization: clear list to prevent repeat processing
          allCombo.put(key, new ArrayList<>());
        }
      }
      return 0;
    }
```

(Note: Pair can be AbstractMap.SimpleEntry or custom pair class in Java.)

Complexity

- Time: O(M * N) where M = length of word, N = number of words (preprocessing & BFS neighbors). Using bidirectional BFS improves practically.
- Space: O(M * N)

Example
begin=hit, end=cog, wordList=[hot, dot, dog, lot, log, cog] → length = 5 (hit→hot→dot→dog→cog).

---

38) Cheapest Flights Within K Stops

Problem
Given n nodes and list of edges (u,v,price), source, destination, and K (max stops), find cheapest price from source to destination with at most K stops. If no such route, return -1.

What interviewer tests
Graph shortest path variant with bounded hops — knowledge of Bellman-Ford relaxation per hop or modified Dijkstra retaining hop counts.

Approach (Bellman-Ford style DP by hops)
Maintain dist[k][v] = cheapest price to reach v using at most k edges. Iteratively relax edges up to K+1 times.

Algorithm (Bellman-Ford optimized)

1. Initialize dist[] with INF, dist[src] = 0.
2. For i = 0..K:
    - copy tmp = dist.clone()
    - for each edge (u,v,w): tmp[v] = min(tmp[v], dist[u] + w)
    - dist = tmp
3. If dist[dst] is INF return -1 else return value.

Java

```
public int findCheapestPrice(int n, int[][] flights, int src, int dst, int K) {
  final int INF = Integer.MAX_VALUE / 2;
  int[] dist = new int[n];
  Arrays.fill(dist, INF);
  dist[src] = 0;

  for (int i = 0; i <= K; i++) {
    int[] tmp = Arrays.copyOf(dist, n);
    for (int[] f : flights) {
      int u = f[0], v = f[1], w = f[2];
      if (dist[u] + w < tmp[v]) {
        tmp[v] = dist[u] + w;
      }
    }
    dist = tmp;
  }
```

```
    return dist[dst] >= INF ? -1 : dist[dst];
}
```

Complexity

- Time: O(K * E) where E = number of flights
- Space: O(n)

Example
n=3, flights=[[0,1,100],[1,2,100],[0,2,500]], src=0,dst=2,K=1 → cheapest = 200 via 0→1→2.

---

39) Connected Components in Undirected Graph

Problem
Given n nodes (0..n-1) and edge list, return number of connected components.

What interviewer tests
DFS/BFS/Union-Find (disjoint set union) application.

Approach (Union-Find or DFS)
Union-Find is concise: initially n components, union endpoints of each edge; resulting unique parents count is component count.

Algorithm (Union-Find)

1. Initialize parent[i]=i, rank[] optional.
2. For each edge(u,v) do union(u,v).
3. Count distinct roots.

Java

```
public int countComponents(int n, int[][] edges) {
    int[] parent = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
    int count = n;

    for (int[] e : edges) {
```

```
      int a = find(parent, e[0]);
      int b = find(parent, e[1]);
      if (a != b) {
        parent[a] = b;
        count--;
      }
    }
    return count;
}
private int find(int[] parent, int x) {
    if (parent[x] != x) parent[x] = find(parent, parent[x]);
    return parent[x];
}
```

Complexity

- Time: approximately O((V + E) * α(V)) due to union-find amortized inverse Ackermann factor.
- Space: O(V)

Example
n=5, edges=[[0,1],[1,2],[3,4]] → components = 2 ({0,1,2} and {3,4}).

---

40) Detect Cycle in Directed Graph

Problem
Given a directed graph, determine if it contains a cycle.

What interviewer tests
DFS with coloring states (white/gray/black) or DFS recursion stack detection; understanding of directed cycles vs undirected cycle detection.

Approach (DFS with three-state coloring)
During DFS, mark node GRAY when entering, BLACK when leaving. If you encounter a GRAY neighbor, there's a cycle.

Algorithm

1. state[]: 0 = unvisited, 1 = visiting (gray), 2 = visited (black).
2. For each node if unvisited call dfs(u):
    - set state[u] = 1
    - for v in adj[u]:
        - if state[v] == 0 and dfs(v) return true
        - else if state[v] == 1 return true
    - state[u] = 2
    - return false

Java

```
public boolean hasCycleDirected(int n, List<List<Integer>> adj) {
  int[] state = new int[n];
  for (int i = 0; i < n; i++) {
    if (state[i] == 0) {
      if (dfsCycle(i, adj, state)) return true;
    }
  }
  return false;
}

private boolean dfsCycle(int u, List<List<Integer>> adj, int[] state) {
  state[u] = 1; // visiting
  for (int v : adj.get(u)) {
    if (state[v] == 0) {
```

```
        if (dfsCycle(v, adj, state)) return true;
      } else if (state[v] == 1) {
        return true; // found back edge
      }
    }
    state[u] = 2; // visited
    return false;
}
```

Complexity

- Time: O(V + E)
- Space: O(V) recursion stack and state array

Example
Edges: 0→1, 1→2, 2→0 → cycle exists.

41) Merge K Sorted Lists

Problem
Given an array of k sorted linked lists, merge them into one sorted linked list and return its head.

What interviewer tests

- Use of a heap to merge multiple sorted streams efficiently
- Ability to reason about time/space trade-offs (heap vs pairwise merging)
- Linked-list pointer handling

Approach (optimal)
Use a min-heap (PriorityQueue) keyed by node value. Put the head of each non-empty list into the heap. Repeatedly extract the smallest node, append it to the result list, and if the extracted node has a next, push that next into the heap.

Why this is good

Each insertion/extraction is O(log k). Total nodes = N (sum of lengths). Time = O(N log k), which is optimal for this approach.

Algorithm (step-by-step)

1. Create a min-heap comparing node values.
2. For each list head, if not null, add to heap.
3. Create dummy tail. While heap not empty:
    - pop smallest node n
    - tail.next = n; tail = tail.next
    - if n.next != null, push n.next into heap
4. Return dummy.next

Java code

```
public ListNode mergeKLists(ListNode[] lists) {
  PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> Integer.compare(a.val, b.val));
  for (ListNode head : lists) {
    if (head != null) pq.offer(head);
  }
  ListNode dummy = new ListNode(0), tail = dummy;
  while (!pq.isEmpty()) {
    ListNode node = pq.poll();
    tail.next = node;
    tail = tail.next;
    if (node.next != null) pq.offer(node.next);
  }
  return dummy.next;
}
```

Complexity

- Time: O(N log k) where N = total nodes, k = number of lists.
- Space: O(k) for the heap (plus O(1) extra for pointers).

Example
lists = [[1,4,5],[1,3,4],[2,6]] → merged: [1,1,2,3,4,4,5,6].

---

42) Find Median from Data Stream

Problem
Design a data structure that supports these two operations:

- void addNum(int num) — add a number to the data stream.
- double findMedian() — return the median of all elements so far.

What interviewer tests

- Two-heap technique for streaming median
- Careful maintenance of balance invariants and edge cases (even/odd count)
- Understanding of amortized complexity

Approach (optimal)
Maintain two heaps:

- maxHeap (lower half) — a max-heap storing smaller half of numbers.
- minHeap (upper half) — a min-heap storing larger half of numbers.

Invariants:

- maxHeap.size() == minHeap.size() or maxHeap.size() == minHeap.size() + 1
- All elements in maxHeap ≤ all elements in minHeap.

When adding:

1. Add to maxHeap first (or compare to root), then move top to minHeap to maintain order.
2. Rebalance sizes: if minHeap has more elements, move one to maxHeap. If maxHeap has more than one extra element, move one to minHeap.

Median:

- If total count odd → top of maxHeap
- If even → average of tops of both heaps

Java code

```
class MedianFinder {
  private PriorityQueue<Integer> maxHeap; // lower half (max-heap)
  private PriorityQueue<Integer> minHeap; // upper half (min-heap)

  public MedianFinder() {
    maxHeap = new PriorityQueue<>(Collections.reverseOrder());
    minHeap = new PriorityQueue<>();
  }

  public void addNum(int num) {
    // add to maxHeap first
    maxHeap.offer(num);
    // ensure every element in maxHeap <= every element in minHeap
    if (!maxHeap.isEmpty() && !minHeap.isEmpty() && maxHeap.peek() > minHeap.peek())
{
      minHeap.offer(maxHeap.poll());
    }
    // rebalance sizes: maxHeap can have at most 1 more
    if (maxHeap.size() < minHeap.size()) {
      maxHeap.offer(minHeap.poll());
    } else if (maxHeap.size() - minHeap.size() > 1) {
      minHeap.offer(maxHeap.poll());
```

```
    }
  }

  public double findMedian() {
    if (maxHeap.isEmpty()) throw new IllegalStateException("No elements");
    if (maxHeap.size() > minHeap.size()) {
      return (double) maxHeap.peek();
    } else {
      return ((double) maxHeap.peek() + (double) minHeap.peek()) / 2.0;
    }
  }
}
```

Complexity

- addNum: O(log n) (heap operations)
- findMedian: O(1)
- Space: O(n) for stored elements

Example
Add: 1 → median 1
Add: 2 → median (1+2)/2 = 1.5
Add: 3 → median 2

---

43) K Closest Points to Origin

Problem
Given an array of points points[i] = [xi, yi], return the k closest points to the origin (0,0) based on Euclidean distance. You may return the answer in any order.

What interviewer tests

- Use of heap to select top-k elements efficiently

- Custom comparator and squared-distance avoidance of expensive sqrt
- Time/space trade-offs (min-heap vs max-heap of size k)

Approach (optimal for large N, small k)
Use a max-heap (size k) so the heap keeps the k closest points seen so far. For each point, compute squared distance d = x*x + y*y. Push point onto heap; if heap size > k, poll the farthest (heap root). At the end, heap contains k closest points.

Algorithm (step-by-step)

1. Create a max-heap comparing by squared distance descending.
2. For each point:
    - compute d = x*x + y*y
    - push into heap
    - if heap.size() > k, poll
3. Collect points from heap into result list.

Java code

```
public int[][] kClosest(int[][] points, int k) {
  PriorityQueue<int[]> maxHeap = new PriorityQueue<>(
    (a, b) -> Long.compare((long)b[0]*b[0] + (long)b[1]*b[1],
                (long)a[0]*a[0] + (long)a[1]*a[1])
  );

  for (int[] p : points) {
    maxHeap.offer(p);
    if (maxHeap.size() > k) maxHeap.poll();
  }

  int[][] res = new int[k][2];
  for (int i = k - 1; i >= 0; i--) {
```

```
    res[i] = maxHeap.poll();
  }
  return res;
}
```

Complexity

- Time: O(N log k) where N = number of points
- Space: O(k) for heap

Example
points = [[1,3],[-2,2]], k = 1 → distances 10 and 8 → return [-2,2].

---

44) Top K Frequent Words

Problem
Given a list of words, return the k most frequent words. Sort by frequency descending. If ties, sort lexicographically (alphabetical).

What interviewer tests

- Combining hashmap counting with custom heap comparator
- Tie-breaking rules implementation
- Stable extraction and result ordering

Approach (hashmap + heap or bucket sort)
Use HashMap to count frequencies. Use a min-heap of size k where comparator orders by frequency ascending, and if frequency equal, by lexicographic descending (so the word that should be evicted first is lexicographically larger when frequencies tie). After processing counts into heap, pop to build result in reverse order.

Algorithm (step-by-step)

1. Count frequencies in Map<String, Integer> freq.
2. Create a min-heap with comparator: (a,b) → if freq[a] != freq[b] return freq[a]-freq[b]; else return b.compareTo(a) (reverse lexicographic).
3. For each word in map: push into heap; if heap.size() > k poll.

4. Pop elements into list and reverse to get descending ordering.

Java code

```
public List<String> topKFrequentWords(String[] words, int k) {
   Map<String, Integer> freq = new HashMap<>();
   for (String w : words) freq.put(w, freq.getOrDefault(w, 0) + 1);

   PriorityQueue<String> pq = new PriorityQueue<>((a, b) -> {
      int fa = freq.get(a), fb = freq.get(b);
      if (fa != fb) return Integer.compare(fa, fb); // smaller freq first
      return b.compareTo(a); // for same freq, lexicographically larger comes first to be popped earlier
   });

   for (String w : freq.keySet()) {
      pq.offer(w);
      if (pq.size() > k) pq.poll();
   }

   List<String> res = new ArrayList<>();
   while (!pq.isEmpty()) res.add(pq.poll());
   Collections.reverse(res); // highest freq, lexicographically smallest first
   return res;
}
```

Complexity

- Time: O(N log k) where N = number of unique words (or total words for counting)
- Space: O(N) for map and O(k) for heap

Example
words = ["i","love","leetcode","i","love","coding"], k = 2 → counts: i:2, love:2, coding:1, leetcode:1
Tie between "i" and "love" → lexicographically "i" < "love" → result ["i","love"].

45) Climbing Stairs

Problem
You are climbing a staircase with n steps. Each time you can climb either 1 or 2 steps. How many distinct ways can you climb to the top?

What interviewer tests
Basic DP / recurrence recognition; ability to turn recurrence into O(1) space iterative solution.

Intuition & Approach
This is Fibonacci-like: ways[n] = ways[n-1] + ways[n-2]. Base: ways[0]=1 (one way to stay), ways[1]=1.

Algorithm (bottom-up)
Use two variables to iteratively compute the next Fibonacci number.

Java

```
public int climbStairs(int n) {
   if (n <= 1) return 1;
   int a = 1; // ways[0]
   int b = 1; // ways[1]
   for (int i = 2; i <= n; i++) {
      int next = a + b;
      a = b;
      b = next;
   }
   return b;
}
```

Complexity

- Time: O(n)
- Space: O(1)

Example
n=3 → ways: 3 ([1+1+1], [1+2], [2+1]).

---

46) House Robber

Problem
Given a list of non-negative integers nums representing money at each house in a row, compute the maximum amount you can rob without robbing adjacent houses.

What interviewer tests
DP for optimal choices with local constraints; space optimization.

Intuition & Approach
At each house i: dp[i] = max(dp[i-1], dp[i-2] + nums[i]). You either skip current (take dp[i-1]) or rob current (dp[i-2] + nums[i]).

Algorithm (iterative, O(1) space)
Keep prev1 = dp[i-1], prev2 = dp[i-2].

Java

```
public int rob(int[] nums) {
  if (nums == null || nums.length == 0) return 0;
  int prev2 = 0; // dp[i-2]
  int prev1 = 0; // dp[i-1]
  for (int val : nums) {
    int curr = Math.max(prev1, prev2 + val);
    prev2 = prev1;
    prev1 = curr;
  }
  return prev1;
}
```

Complexity

- Time: O(n)
- Space: O(1)

Example
nums=[2,7,9,3,1] → optimal = 12 (2 + 9 + 1).

---

47) Longest Increasing Subsequence (LIS)

Problem
Given an integer array nums, return the length of the longest strictly increasing subsequence.

What interviewer tests
DP basics and knowledge of the O(n log n) patience-sorting / binary-search optimization.

Approach 1 (DP $O(n^2)$)
dp[i] = length of LIS ending at i. For each i, check j < i with nums[j] < nums[i], dp[i] = max(dp[i], dp[j]+1).

Approach 2 (Optimal O(n log n)) — patience method
Maintain an array tails[] where tails[len] = smallest possible tail value of an increasing subsequence of length len+1. For each num, binary-search tails to replace first >= num.

Java (O(n log n))

```
public int lengthOfLIS(int[] nums) {
  if (nums == null || nums.length == 0) return 0;
  int[] tails = new int[nums.length];
  int size = 0;
  for (int x : nums) {
    int i = Arrays.binarySearch(tails, 0, size, x);
    if (i < 0) i = -(i + 1); // insertion point
    tails[i] = x;
    if (i == size) size++;
```

```
    }
    return size;
}
```

Complexity

- Time: O(n log n) (optimal)
- Space: O(n)

Example
nums = [10,9,2,5,3,7,101,18] → LIS length = 4 (e.g., [2,3,7,101]).

---

48) Coin Change (Minimum Coins)

Problem
Given coin denominations coins[] and an amount amount, compute the fewest number of coins needed to make up that amount. Return -1 if impossible.

What interviewer tests
Classic unbounded knapsack DP; ability to choose correct DP direction (amount outer loop or coin outer loop) and initialize properly.

Approach (Bottom-up DP)
dp[a] = min coins to make amount a. Initialize dp[0] = 0, others = INF. For each coin, for a = coin .. amount: dp[a] = min(dp[a], dp[a-coin] + 1).

Java

```
public int coinChange(int[] coins, int amount) {
    int INF = amount + 1;
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, INF);
    dp[0] = 0;
    for (int coin : coins) {
        for (int a = coin; a <= amount; a++) {
            dp[a] = Math.min(dp[a], dp[a - coin] + 1);
```

```
    }
  }
  return dp[amount] > amount ? -1 : dp[amount];
}
```

Complexity

- Time: O(amount * numCoins)
- Space: O(amount)

Example
coins = [1,2,5], amount = 11 → dp[11]=3 (5+5+1).

---

49) Partition Equal Subset Sum

Problem
Given a non-empty array nums, determine if the array can be partitioned into two subsets with equal sum.

What interviewer tests
Reduction to subset-sum; 0/1 knapsack boolean DP.

Approach
Sum all numbers; if odd, return false. Target = sum/2. Use boolean DP dp[a] indicates whether a sum achievable. Iterate numbers and update dp backwards (to avoid reuse): for a = target .. num: dp[a] |= dp[a - num].

Java

```
public boolean canPartition(int[] nums) {
  int total = 0;
  for (int n : nums) total += n;
  if ((total & 1) == 1) return false;
  int target = total / 2;
  boolean[] dp = new boolean[target + 1];
  dp[0] = true;
```

```
  for (int num : nums) {
    for (int a = target; a >= num; a--) {
      dp[a] = dp[a] || dp[a - num];
    }
  }
  return dp[target];
}
```

Complexity

- Time: O(n * target)
- Space: O(target)

Example
nums = [1,5,11,5] → total=22 target=11 → true ([11] vs [1,5,5]).

---

50) Longest Common Subsequence (LCS)

Problem
Given two strings text1 and text2, return the length of their longest common subsequence (not substring). A subsequence need not be contiguous.

What interviewer tests
2D DP formulation and ability to derive recurrence.

Approach
Let dp[i][j] = LCS length of text1[0..i-1] and text2[0..j-1]. Recurrence:

- if text1[i-1] == text2[j-1] → dp[i][j] = dp[i-1][j-1] + 1
- else dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1])

Use 2D table or optimize to 1D (rolling array).

Java (2D)

```
public int longestCommonSubsequence(String t1, String t2) {
  int n = t1.length(), m = t2.length();
```

```
    int[][] dp = new int[n + 1][m + 1];
    for (int i = 1; i <= n; i++) {
      for (int j = 1; j <= m; j++) {
        if (t1.charAt(i - 1) == t2.charAt(j - 1)) {
          dp[i][j] = dp[i - 1][j - 1] + 1;
        } else {
          dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
        }
      }
    }
    return dp[n][m];
}
```

Complexity

- Time: O(n * m)
- Space: O(n * m) (can reduce to O(min(n,m)) by rolling array)

Example
t1 = "abcde", t2 = "ace" → LCS = "ace", length = 3.

---

51) Word Break

Problem
Given a string s and a dictionary of words wordDict, determine if s can be segmented into a space-separated sequence of dictionary words.

What interviewer tests
DP on strings + substring checks; ability to optimize by checking only valid word lengths.

Approach
Let dp[i] be boolean: s[0..i-1] can be segmented. dp[0] = true. For each i from 1..n: for each word in dict (or j from 0..i-1), if dp[j] true and s.substring(j,i) in dict → set dp[i]=true.

Optimizations

- Use HashSet for dictionary for O(1) lookup.
- Optionally restrict candidate lengths by min/max word length.

Java

```
public boolean wordBreak(String s, List<String> wordDict) {
   Set<String> dict = new HashSet<>(wordDict);
   int n = s.length();
   boolean[] dp = new boolean[n + 1];
   dp[0] = true;
   for (int i = 1; i <= n; i++) {
      for (int j = 0; j < i; j++) {
         if (dp[j] && dict.contains(s.substring(j, i))) {
            dp[i] = true;
            break;
         }
      }
   }
   return dp[n];
}
```

Complexity

- Time: $O(n^2 * k)$ if substring creation costs k; practically $O(n^2)$ with optimizations.
- Space: O(n)

Example
s = "leetcode", wordDict = ["leet","code"] → true.

52) Implement Trie (Prefix Tree)

Problem
Implement a Trie with insert(String word), search(String word) returning boolean (exact word), and startsWith(String prefix) returning boolean (any word starts with prefix).

What interviewer tests
Designing a tree-like character-indexed data structure, memory vs time trade-offs, understanding prefix operations useful for autocomplete and dictionary problems.

Approach & Intuition
A Trie node holds an array/map of children (for each character) and a boolean flag isWord marking word termination. For ASCII/lowercase-only inputs, use array size 26 for speed and memory predictability; otherwise use Map<Character, TrieNode>.

Algorithm (operations)

- insert: walk/create nodes for each char, mark last node .isWord = true.
- search: traverse nodes for each char; if traversal finishes and isWord true → found.
- startsWith: traverse nodes for prefix; if traversal succeeds → true.

Java (lowercase a–z optimized)

```
class Trie {
  private static class Node {
    Node[] children = new Node[26];
    boolean isWord = false;
  }

  private final Node root = new Node();

  public void insert(String word) {
    Node cur = root;
    for (char c : word.toCharArray()) {
      int idx = c - 'a';
      if (cur.children[idx] == null) cur.children[idx] = new Node();
```

```
            cur = cur.children[idx];
        }
        cur.isWord = true;
    }

    public boolean search(String word) {
        Node cur = root;
        for (char c : word.toCharArray()) {
            cur = cur.children[c - 'a'];
            if (cur == null) return false;
        }
        return cur.isWord;
    }

    public boolean startsWith(String prefix) {
        Node cur = root;
        for (char c : prefix.toCharArray()) {
            cur = cur.children[c - 'a'];
            if (cur == null) return false;
        }
        return true;
    }
}
```

Complexity

- insert, search, startsWith: Time O(L) where L = length of word/prefix.
- Space: O(sum of lengths of inserted words) for trie nodes.

Example
Insert "apple", search("apple") → true, search("app") → false, startsWith("app") → true.

---

53) Word Search II (Trie + Backtracking)

Problem
Given a 2D board of characters and a list of words, find all words that appear in the board. Words are formed by sequentially adjacent cells (horizontally or vertically), and the same cell cannot be used more than once per word.

What interviewer tests
Combining Tries for prefix pruning with DFS/backtracking over a grid — performance optimization and pruning to avoid TLE on large dictionaries/boards.

Approach & Intuition
Build a Trie for word list. For each cell in board, start DFS using current char; as you traverse, check whether current path prefix exists in Trie — if not, prune. When you hit a Trie node with isWord=true, add to result and optionally mark it false to avoid duplicates.

Algorithm (high-level)

1. Build Trie of words.
2. For each board cell (i,j), start dfs(i,j, trieRoot, pathBuilder).
3. In DFS:
    - Check bounds & visited marker; char c = board[i][j]; go to node = node.children[c]; if null → backtrack.
    - Append char; if node.isWord true → add word to result and set node.isWord=false (dedupe).
    - Mark visited (e.g., set board[i][j] = '#' or use visited boolean), recurse to 4 neighbors.
    - Restore visited state and pop char.

Java (sketch, careful about string building and Trie node carrying word)

```
class TrieNode {
  TrieNode[] children = new TrieNode[26];
  String word = null; // store full word at terminal node for easy result add
```

```
}

public List<String> findWords(char[][] board, String[] words) {
   TrieNode root = buildTrie(words);
   List<String> res = new ArrayList<>();
   int m = board.length, n = board[0].length;
   for (int i = 0; i < m; i++) {
      for (int j = 0; j < n; j++) {
         dfs(board, i, j, root, res);
      }
   }
   return res;
}

private void dfs(char[][] board, int i, int j, TrieNode node, List<String> res) {
   if (i < 0 || j < 0 || i >= board.length || j >= board[0].length) return;
   char c = board[i][j];
   if (c == '#' || node.children[c - 'a'] == null) return;
   node = node.children[c - 'a'];
   if (node.word != null) {
      res.add(node.word);
      node.word = null; // avoid duplicate
   }
   board[i][j] = '#';
   dfs(board, i+1, j, node, res);
   dfs(board, i-1, j, node, res);
```

```
    dfs(board, i, j+1, node, res);
    dfs(board, i, j-1, node, res);
    board[i][j] = c;
}

private TrieNode buildTrie(String[] words) {
    TrieNode root = new TrieNode();
    for (String w : words) {
        TrieNode cur = root;
        for (char ch : w.toCharArray()) {
            int idx = ch - 'a';
            if (cur.children[idx] == null) cur.children[idx] = new TrieNode();
            cur = cur.children[idx];
        }
        cur.word = w; // store word at the terminal
    }
    return root;
}
```

Complexity

- Time: Upper-bounded by O(M * 4^L) where M = number of cells and L = max word length, but Trie pruning reduces practical cost.
- Space: O(sum of word lengths) for Trie + recursion stack.

Example

Board:

o a a n

e t a e

i h k r

i f l v

Words: ["oath","pea","eat","rain"] → found ["oath","eat"].

---

54) Design Search Autocomplete System (Trie + Ranking)

Problem
Design an autocomplete system that, given a prefix, returns top-k suggestions sorted by frequency (and perhaps lexicographically for ties). Implement insert/update and query operations.

What interviewer tests
Combining Trie for prefix retrieval with auxiliary ranking data (min-heaps or sorted lists) stored at nodes; trade-offs between precomputation (store top-k at each node) vs runtime computation.

Approach & Intuition
Two common patterns:

1. **Precompute**: At each Trie node store a list of top-k suggestions (word + freq). Query becomes O(L + k). Insert/update requires updating lists up the node path (costlier).
2. **On-demand**: Traverse to prefix node and run DFS to collect all words under it, then sort/select top-k (slower for large subtree).

For interview: describe both; implement a simple version using Trie + storing Map<String, Integer> counts at terminal nodes and at query time do DFS gather then use a heap to extract top-k.

Sketch Implementation (simpler on-demand with heap)

```
class Autocomplete {
  TrieNode root = new TrieNode();

  public void insert(String word, int freq) {
    TrieNode cur = root;
    for (char c : word.toCharArray()) {
      if (cur.children[c - 'a'] == null) cur.children[c - 'a'] = new TrieNode();
```

```
        cur = cur.children[c - 'a'];
    }
    cur.word = word;
    cur.freq = freq; // store frequency at terminal
}

public List<String> query(String prefix, int k) {
    TrieNode cur = root;
    for (char c : prefix.toCharArray()) {
        cur = cur.children[c - 'a'];
        if (cur == null) return Collections.emptyList();
    }
    // collect words under cur
    PriorityQueue<TrieNode> heap = new PriorityQueue<>((a,b) -> {
        if (a.freq != b.freq) return Integer.compare(a.freq, b.freq); // min-heap by freq
        return b.word.compareTo(a.word); // tie-break reverse lexicographic for eviction
    });
    collect(cur, heap, k);
    List<String> res = new ArrayList<>();
    while (!heap.isEmpty()) res.add(heap.poll().word);
    Collections.reverse(res); // highest freq first
    return res;
}

private void collect(TrieNode node, PriorityQueue<TrieNode> heap, int k) {
    if (node == null) return;
```

```
        if (node.word != null) {
            heap.offer(node);
            if (heap.size() > k) heap.poll();
        }
        for (TrieNode ch : node.children) {
            if (ch != null) collect(ch, heap, k);
        }
    }

    static class TrieNode {
        TrieNode[] children = new TrieNode[26];
        String word = null;
        int freq = 0;
    }
}
```

Complexity

- Query (on-demand): O(#nodes in subtree + log k) worst-case — can be expensive.
- Precompute approach: Query O(L + k) with higher insert/update costs.

Example
Insert: ("apple",5), ("app",3), ("apricot",2). Query "ap", k=2 → ["apple","app"].

🔷 CONCURRENCY & LOW-LEVEL DESIGN — (4 problems)

---

55) Producer–Consumer (BlockingQueue / wait-notify)

Problem
Implement a thread-safe producer-consumer pattern where producers add items to a

bounded buffer and consumers remove them. Ensure correct blocking behavior when buffer is full/empty.

What interviewer tests
Concurrency primitives, using high-level abstractions vs low-level wait/notify, deadlock avoidance, correctness vs liveness trade-offs.

Approach & Intuition
Prefer using Java's built-in BlockingQueue (ArrayBlockingQueue, LinkedBlockingQueue) for production-grade correctness. For interview, show both usage and an understanding of implementing it with wait()/notify().

Implementation A — Using BlockingQueue (recommended)

```
class Producer implements Runnable {
  private final BlockingQueue<Integer> queue;
  public Producer(BlockingQueue<Integer> q) { this.queue = q; }
  @Override
  public void run() {
    try {
      for (int i = 0; i < 100; i++) {
        queue.put(i); // blocks if queue is full
        System.out.println("Produced " + i);
      }
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
  }
}

class Consumer implements Runnable {
  private final BlockingQueue<Integer> queue;
  public Consumer(BlockingQueue<Integer> q) { this.queue = q; }
  @Override
```

```
  public void run() {
    try {
      while (true) {
        Integer item = queue.take(); // blocks if empty
        System.out.println("Consumed " + item);
      }
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
  }
}
```

Usage:

```
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(10);
new Thread(new Producer(q)).start();
new Thread(new Consumer(q)).start();
```

Implementation B — wait()/notify() (educational)
Use synchronized on shared buffer with wait() when full/empty and notifyAll() on state changes. Be careful to use while-loops to recheck conditions (spurious wakeups).

Complexity & Notes

- Correctness: BlockingQueue handles fairness and concurrency; wait/notify is error-prone but demonstrates understanding.
- Avoid calling notify() alone in some cases; prefer notifyAll() unless you reason about precise conditions.

Example
Multiple producers and consumers coordinate smoothly with BlockingQueue.

---

56) Thread-safe Singleton / Double-checked Locking

Problem
Implement a thread-safe singleton in Java with lazy initialization and minimal synchronization overhead.

What interviewer tests
Understanding of concurrency, volatile semantics, double-checked locking pitfalls, and Java Memory Model.

Approach & Intuition
Use the Bill Pugh singleton (Initialization-on-demand holder) or double-checked locking with volatile.

Implementation A — Initialization-on-demand Holder (best)

```
public class Singleton {
  private Singleton() {}

  private static class Holder {
    static final Singleton INSTANCE = new Singleton();
  }

  public static Singleton getInstance() {
    return Holder.INSTANCE;
  }
}
```

This is lazy and thread-safe (class loading guarantees).

Implementation B — Double-checked locking with volatile

```
public class Singleton {
  private static volatile Singleton instance;
  private Singleton() {}

  public static Singleton getInstance() {
    if (instance == null) {
      synchronized (Singleton.class) {
```

```
            if (instance == null) {
                instance = new Singleton();
            }
        }
    }
    return instance;
  }
}
```

Complexity & Notes

- Use volatile to prevent reordering issues (constructor escape).
- Holder pattern is simpler and preferred.

---

57) Design a Rate Limiter (Token Bucket)

Problem
Design an in-memory rate limiter supporting allowRequest(userId) that returns true if the user is allowed based on a limit of R requests per T seconds (e.g., 100 requests per 60s). Discuss token-bucket approach.

What interviewer tests
System design at small scale: moving-window counters vs token bucket vs leaky bucket, concurrency, correctness under bursts, memory trade-offs.

Approach (Token Bucket)

- Each user has a bucket with capacity maxTokens and refill rate r = maxTokens / T tokens per second.
- On request: compute tokens to add since last refill: newTokens = (now - lastRefill) * r. tokens = min(maxTokens, tokens + newTokens). If tokens >= 1 → allow and tokens -= 1; else reject.

Algorithm (thread-safe, approximate)

- Maintain a ConcurrentHashMap<userId, Bucket> where Bucket has double tokens, long lastRefillTimestamp.

- For each allowRequest:
    - get bucket (create if absent with atomic put-if-absent)
    - synchronized on bucket or use AtomicReference to update tokens atomically
    - compute refill and decide

Java (sketch, per-user locking)

```
class TokenBucket {
  private final double capacity;
  private final double refillRatePerMillis;
  private double tokens;
  private long lastRefillMillis;

  TokenBucket(double capacity, long windowMillis) {
    this.capacity = capacity;
    this.refillRatePerMillis = capacity / (double) windowMillis;
    this.tokens = capacity;
    this.lastRefillMillis = System.currentTimeMillis();
  }

  synchronized boolean allowRequest() {
    long now = System.currentTimeMillis();
    double added = (now - lastRefillMillis) * refillRatePerMillis;
    tokens = Math.min(capacity, tokens + added);
    lastRefillMillis = now;
    if (tokens >= 1.0) {
      tokens -= 1.0;
      return true;
```

```
    }
    return false;
  }
}

class RateLimiter {
  private final ConcurrentHashMap<String, TokenBucket> buckets = new
ConcurrentHashMap<>();
  private final double capacity;
  private final long windowMillis;

  RateLimiter(double capacity, long windowMillis) {
    this.capacity = capacity;
    this.windowMillis = windowMillis;
  }

  public boolean allowRequest(String userId) {
    TokenBucket bucket = buckets.computeIfAbsent(userId, id -> new
TokenBucket(capacity, windowMillis));
    return bucket.allowRequest();
  }
}
```

Complexity & Notes

- Per-request time: O(1) amortized.
- Space: O(number of active users).
- For distributed systems: need centralized store (Redis) or consistent hashing + local tokens with occasional sync.

Example
R=100 per 60,000 ms → capacity=100, refillRatePerMillis = 100/60000.

---

58) Implement a Simple In-memory Message Queue (Pub/Sub basics)

Problem
Implement a simple in-memory message queue that supports publish(topic, message) and subscribe(topic, Consumer) semantics for a single-process application. Ensure thread-safety, durability in memory, and basic backpressure handling.

What interviewer tests
Understanding of pub/sub primitives, thread-safe queues per topic, consumer handling, design trade-offs (push vs pull, persistence vs in-memory), and supervision for slow consumers.

Approach & Intuition

- Use a ConcurrentHashMap<String, Topic> mapping topic → topic state.
- Each Topic contains a BlockingQueue<Message> or LinkedBlockingDeque.
- publish pushes to queue.
- subscribe starts a consumer thread that polls queue (push to consumer) or registers consumer callback and a worker thread dispatches messages.
- For multiple subscribers: either broadcast (each subscriber has its own queue) or fan-out from central queue.

Implementation (broadcast model: each subscriber has own queue)

- On subscribe(topic, consumer): create LinkedBlockingQueue for subscriber, spawn dispatcher thread that polls subscriber queue and calls consumer. When publish called, place message into every subscriber's queue (bounded) — if queue full, apply backpressure policy (block, drop oldest, drop new, or throw).

Java (sketch)

```
class Message { String payload; long timestamp; /* other metadata */ }


class Topic {
```

```
  private final CopyOnWriteArrayList<LinkedBlockingQueue<Message>> subscribers =
new CopyOnWriteArrayList<>();


  public void publish(Message msg) {
    for (LinkedBlockingQueue<Message> q : subscribers) {
      // best-effort non-blocking publish with bounded queue
      boolean offered = q.offer(msg);
      if (!offered) {
        // policy: drop oldest to make room
        q.poll();
        q.offer(msg);
      }
    }
  }


  public void subscribe(Consumer<Message> consumer, int queueCapacity) {
    LinkedBlockingQueue<Message> q = new LinkedBlockingQueue<>(queueCapacity);
    subscribers.add(q);
    // start a dispatcher thread for this subscriber
    Thread t = new Thread(() -> {
      try {
        while (true) {
          Message m = q.take(); // blocks
          consumer.accept(m);
        }
      } catch (InterruptedException e) {
```

```
            Thread.currentThread().interrupt();
        }
      });
      t.start();
    }
}

class SimplePubSub {
    private final ConcurrentHashMap<String, Topic> topics = new ConcurrentHashMap<>();

    public void publish(String topicName, Message msg) {
      topics.computeIfAbsent(topicName, t -> new Topic()).publish(msg);
    }

    public void subscribe(String topicName, Consumer<Message> consumer, int
queueCapacity) {
      topics.computeIfAbsent(topicName, t -> new Topic()).subscribe(consumer,
queueCapacity);
    }
}
```

Complexity & Notes

- Publish cost: O(subscribers) due to copying to each queue. For many subscribers, use broadcast optimizations or a single log with per-subscriber offsets.
- For durability and scaling: use persistent logs (Kafka), partitioning, and consumer offsets storage (Zookeeper/coordination or Kafka internal offsets).

Example
Publish messages to topic "orders", two subscribers each receive their own stream of messages via their dispatcher threads.